

Chapter 1

Delay Costs Calculation Documentation

1.1 Introduction

The `delay_costs.py` script calculates the costs incurred during the delay period (permitting/regulatory delays) for a transmission line project. These costs include legal, administrative, labor, materials, regulatory, public relations, project management, and miscellaneous expenses.

1.2 Method Overview

1.2.1 Purpose

This script calculates delay costs by:

1. Loading annual delay cost components from YAML configuration
2. Summing all annual delay cost components
3. Multiplying by the number of delay years to get total nominal cost
4. Calculating present value of annual delay costs

1.2.2 Calculation Approach

The delay cost calculation follows this structure:

$$\text{Total Yearly Delay Cost} = \sum_{\text{components}} \text{Annual Component Cost} \quad (1.1)$$

$$\text{Total Delay Cost} = \text{Total Yearly Delay Cost} \times \text{Delay Years} \quad (1.2)$$

$$\text{Present Value} = \sum_{t=1}^D \frac{\text{Total Yearly Delay Cost}}{(1 + r)^t} \quad (1.3)$$

where D is the number of delay years and r is the real WACC discount rate.

1.2.3 Inputs and Outputs

Inputs:

- Annual delay cost components (legal, admin, labor, materials, regulatory, public relations, project management, miscellaneous)
- Delay years (from project timeline)
- Financing parameters (WACC for present value)

Outputs:

- Total yearly delay cost
- Total nominal delay cost
- Present value of delay costs

1.2.4 Integration with Other Scripts

This script integrates with:

- `smart_loaders.py` — `load_delay_costs`, `load_financing_details`, `load_project_technical_details` (centralized; local wrapper `load_project_technical_details()` returns `delay_years` only)
- `financial_utils.py` — `calculate_present_value`
- `smart_output.py` — `CTCCOutputManager` (`add_delay_costs`, `write_batch_summary`)

1.3 Key Functions

1.3.1 `load_project_technical_details()`

Local wrapper that loads project technical details via `smart_loaders.load_project_technical_details` and returns only `delay_years`.

Returns:

- `delay_year` (float): Number of delay years

1.3.2 main()

Main execution function that orchestrates the delay cost calculation and output.

Workflow:

1. Loads delay cost components from YAML
2. Loads delay years from project technical details
3. Sums all annual delay cost components
4. Calculates total nominal cost (annual cost $\times$ delay years)
5. Calculates present value of annual delay costs
6. Displays results and writes to CSV

Key Logic:

```
1 # Load annual delay cost components
2 legal = delay_cost_df["annual_delay_costs"]["legal"]
3 admin = delay_cost_df["annual_delay_costs"]["admin"]
4 labor = delay_cost_df["annual_delay_costs"]["labor"]
5 material_and_equipment = delay_cost_df["annual_delay_costs"]["
    material_and_equipment"]
6 regulatory = delay_cost_df["annual_delay_costs"]["regulatory"]
7 public_relations = delay_cost_df["annual_delay_costs"]["
    public_relations"]
8 project_management = delay_cost_df["annual_delay_costs"]["
    project_management"]
9 miscellaneous = delay_cost_df["annual_delay_costs"]["miscellaneous"]
10
11 # Sum components
12 total_yearly_delay_cost = (legal + admin + labor +
    material_and_equipment
13                             + regulatory + public_relations +
    project_management
14                             + miscellaneous)
15
16 # Calculate total nominal cost
17 total_delay_cost = total_yearly_delay_cost * delay_year
18
19 # Calculate present value (payments from year 1 for delay_year years
    )
20 total_delay_cost_pv = calculate_present_value(
21     total_yearly_delay_cost,
22     financing.wacc_real,
23     delay_year,
24 )
```

1.4 Example

The following example demonstrates a typical usage of the delay cost calculation:

```
1 # Example: 2-year delay period
2 #
3 # Inputs (from YAML):
4 #   - Legal: $50,000/year
5 #   - Admin: $30,000/year
6 #   - Labor: $100,000/year
7 #   - Materials: $20,000/year
8 #   - Regulatory: $40,000/year
9 #   - Public Relations: $25,000/year
10 #   - Project Management: $80,000/year
11 #   - Miscellaneous: $15,000/year
12 #   - Delay years: 2
13 #   - Real WACC: 3%
14 #
15 # Calculation:
16 #   Total yearly delay cost = $50k + $30k + $100k + $20k + $40k
17 #                           + $25k + $80k + $15k = $360,000/year
18 #   Total nominal cost = $360,000 * 2 = $720,000
19 #   Present value = $360,000 * PV annuity (2 years, 3%) = $697,000
20 #
21 # Output:
22 #   - Total yearly delay cost: $360,000
23 #   - Total nominal cost: $720,000
24 #   - Present value: $697,000
```

Integration Example:

```
1 # When called from ctcc.py:
2 csv_manager = CTCCOutputManager()
3 results = {
4     "total_nominal": total_delay_cost,
5     "total_afudc": 0, # Delay costs are not AFUDC-eligible
6     "total_pv": total_delay_cost_pv,
7 }
8 csv_manager.add_delay_costs(results)
9 csv_manager.write_batch_summary()
```

1.5 Key Implementation Details

1.5.1 Delay Cost Components

The script includes eight delay cost components:

- **Legal:** Legal fees and expenses

- **Admin:** Administrative costs
- **Labor:** Labor costs during delay
- **Material and Equipment:** Material and equipment holding costs
- **Regulatory:** Regulatory compliance costs
- **Public Relations:** Public relations and community engagement
- **Project Management:** Project management overhead
- **Miscellaneous:** Other delay-related expenses

1.5.2 AFUDC Treatment

Delay costs are not AFUDC-eligible because they are operating expenses during the delay period, not construction costs. AFUDC applies only to construction work in progress (CWIP).

1.5.3 Present Value Timing

Delay costs are incurred annually during the delay period (years 1 to delay_year). The present value calculation discounts these annual payments back to the base year.

1.5.4 Simplified Structure

This script has a simpler structure than other cost scripts because delay costs are straightforward: annual costs multiplied by delay years, with no complex terrain adjustments or component-level calculations.